

Daily Sales Loader

Code:

```
import os

import logging
from datetime import datetime, timedelta

import pandas as pd

from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.snowflake.hooks.snowflake import SnowflakeHook
from airflow.exceptions import AirflowException

logger = logging.getLogger(__name__)

def create_table():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        create_table_sql = """
        CREATE TABLE IF NOT EXISTS RAW_SALES (
            order_id INT,
            product VARCHAR,
            quantity INT,
            unit_price FLOAT,
            sale_date DATE,
            region VARCHAR,
            loaded_at TIMESTAMP
        );
        """

        hook.run(create_table_sql)

        logger.info("Table RAW_SALES ready.")

    except Exception as e:

        logger.error(
```

```

        f"Error while creating RAW_SALES table: {str(e)}"
    )

    raise AirflowException(
        f"create_table task failed: {str(e)}"
    )

def clear_todays_data():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        count_query = """
        SELECT COUNT(*)
        FROM RAW_SALES
        WHERE loaded_at::DATE = CURRENT_DATE();
        """

        deleted_rows = hook.get_first(count_query)[0]

        delete_query = """
        DELETE FROM RAW_SALES
        WHERE loaded_at::DATE = CURRENT_DATE();
        """

        hook.run(delete_query)

        logger.info(
            f"Deleted {deleted_rows} rows from RAW_SALES."
        )

    except Exception as e:

        logger.error(
            f"Error while clearing today's data: {str(e)}"
        )

        raise AirflowException(
            f"clear_todays_data task failed: {str(e)}"
        )

def load_csv_to_snowflake():
    try:

```

```

file_path = "data/sales_data.csv"

if not os.path.exists(file_path):
    raise AirflowException(
        f"CSV file not found: {file_path}"
    )

df = pd.read_csv(file_path)

hook = SnowflakeHook(
    snowflake_conn_id="snowflake_default"
)

for _, row in df.iterrows():

    product = str(row['product']).replace("'", "'")
    region = str(row['region']).replace("'", "'")

    insert_sql = f"""
    INSERT INTO RAW_SALES (
        order_id,
        product,
        quantity,
        unit_price,
        sale_date,
        region,
        loaded_at
    )
    VALUES (
        {row['order_id']},
        '{product}',
        {row['quantity']},
        {row['unit_price']},
        '{row['sale_date']}',
        '{region}',
        CURRENT_TIMESTAMP()
    )
    """

    try:

        hook.run(insert_sql)

    except Exception as e:

        logger.error(
            f"Failed inserting order_id "
            f"{row['order_id']}: {str(e)}"

```

```

        )

        raise AirflowException(
            "CSV load failed."
        )

    logger.info(
        f"Loaded {len(df)} rows into RAW_SALES."
    )
except Exception as e:

    logger.error(
        f"Error in load_csv_to_snowflake: {str(e)}"
    )

    raise AirflowException(
        f"load_csv_to_snowflake failed: {str(e)}"
    )

def transform_in_snowflake():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        # Create summary table
        create_summary_table = """
        CREATE TABLE IF NOT EXISTS SALES_SUMMARY (
            region VARCHAR,
            total_orders INT,
            total_revenue FLOAT,
            avg_order_value FLOAT,
            summary_date DATE
        );
        """

        hook.run(create_summary_table)

        logger.info("SALES_SUMMARY table ready.")

        # Idempotency delete
        delete_query = """
        DELETE FROM SALES_SUMMARY
        WHERE summary_date = CURRENT_DATE();
        """

```

```

hook.run(delete_query)

logger.info(
    "Deleted today's existing SALES_SUMMARY rows."
)

# Insert transformed data
insert_query = """
INSERT INTO SALES_SUMMARY (
    region,
    total_orders,
    total_revenue,
    avg_order_value,
    summary_date
)
SELECT
    region,
    COUNT(*),
    SUM(quantity * unit_price),
    AVG(quantity * unit_price),
    CURRENT_DATE()
FROM RAW_SALES
WHERE loaded_at::DATE = CURRENT_DATE()
GROUP BY region;
"""

hook.run(insert_query)

count_query = """
SELECT COUNT(*)
FROM SALES_SUMMARY
WHERE summary_date = CURRENT_DATE();
"""

region_count = hook.get_first(count_query)[0]

logger.info(
    f"{region_count} region summary rows written."
)

except Exception as e:

    logger.error(
        f"Error in transform_in_snowflake: {str(e)}"
    )

    raise AirflowException(
        f"transform_in_snowflake failed: {str(e)}"
    )

```

```

    )

def quality_check():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        row_count_query = """
        SELECT COUNT(*)
        FROM RAW_SALES
        WHERE loaded_at::DATE = CURRENT_DATE();
        """

        row_count = hook.get_first(
            row_count_query
        )[0]

        if row_count == 0:

            logger.error(
                "FAIL: No rows found in RAW_SALES."
            )

            raise AirflowException(
                "Quality Check Failed: No rows loaded."
            )

        logger.info(
            f"PASS: Row count check passed. Rows = {row_count}"
        )

        null_check_query = """
        SELECT COUNT(*)
        FROM RAW_SALES
        WHERE product IS NULL
           OR unit_price IS NULL;
        """

        null_count = hook.get_first(
            null_check_query
        )[0]

        if null_count > 0:

            logger.warning(

```

```

        f"WARNING: Found {null_count} NULL rows."
    )

else:

    logger.info(
        "PASS: NULL check passed."
    )

    negative_price_query = """
    SELECT COUNT(*)
    FROM RAW_SALES
    WHERE unit_price < 0;
    """

    negative_count = hook.get_first(
        negative_price_query
    )[0]

    if negative_count > 0:

        logger.error(
            f"FAIL: Found {negative_count} negative prices."
        )

        raise AirflowException(
            "Quality Check Failed: Negative prices found."
        )

    logger.info(
        "PASS: Negative price check passed."
    )

    logger.info(
        "All quality checks completed successfully."
    )

except Exception as e:

    logger.error(
        f"Error in quality_check: {str(e)}"
    )

    raise AirflowException(
        f"quality_check failed: {str(e)}"
    )

```

```
def print_report():
```

```

try:

    hook = SnowflakeHook(
        snowflake_conn_id='snowflake_default'
    )

    report_query = """
SELECT
    region,
    total_orders,
    total_revenue,
    avg_order_value
FROM SALES_SUMMARY
WHERE summary_date = CURRENT_DATE();
"""

    report_data = hook.get_records(
        report_query
    )

    logger.info(
        "===== SALES REPORT ====="
    )

    grand_total = 0

    for row in report_data:

        region = row[0]
        total_orders = row[1]
        total_revenue = row[2]
        avg_order_value = row[3]

        grand_total += total_revenue

        logger.info(
            f"""
            Region: {region}
            Total Orders: {total_orders}
            Total Revenue: {total_revenue}
            Average Order Value: {avg_order_value}
            """
        )

    logger.info(
        f"GRAND TOTAL REVENUE: {grand_total}"
    )

```



```

        logger.info(
            "===== END REPORT ====="
        )

    except Exception as e:

        logger.error(
            f"Error in print_report: {str(e)}"
        )

        raise AirflowException(
            f"print_report failed: {str(e)}"
        )

default_args = {
    'owner': 'airflow',
    'retries': 2,
    'retry_delay': timedelta(minutes=5)
}

with DAG(
    dag_id='daily_sales_loader',
    schedule='@daily',
    start_date=datetime(2023, 1, 1),
    catchup=False,
    default_args=default_args
) as dag:

    create_table_task = PythonOperator(
        task_id='create_table',
        python_callable=create_table
    )

    clear_todays_data_task = PythonOperator(
        task_id='clear_todays_data',
        python_callable=clear_todays_data
    )

    load_csv_task = PythonOperator(
        task_id='load_csv_to_snowflake',
        python_callable=load_csv_to_snowflake
    )

    transform_task = PythonOperator(
        task_id='transform_in_snowflake',
        python_callable=transform_in_snowflake
    )

```

```

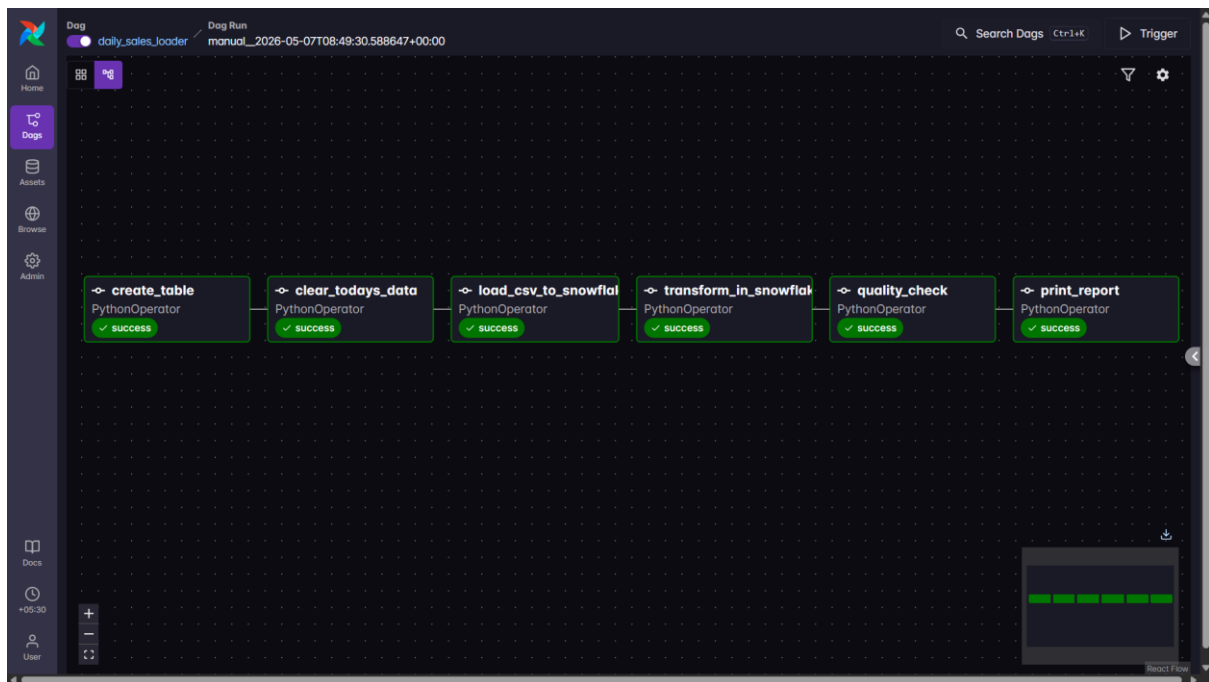
quality_check_task = PythonOperator(
    task_id='quality_check',
    python_callable=quality_check
)

print_report_task = PythonOperator(
    task_id='print_report',
    python_callable=print_report
)

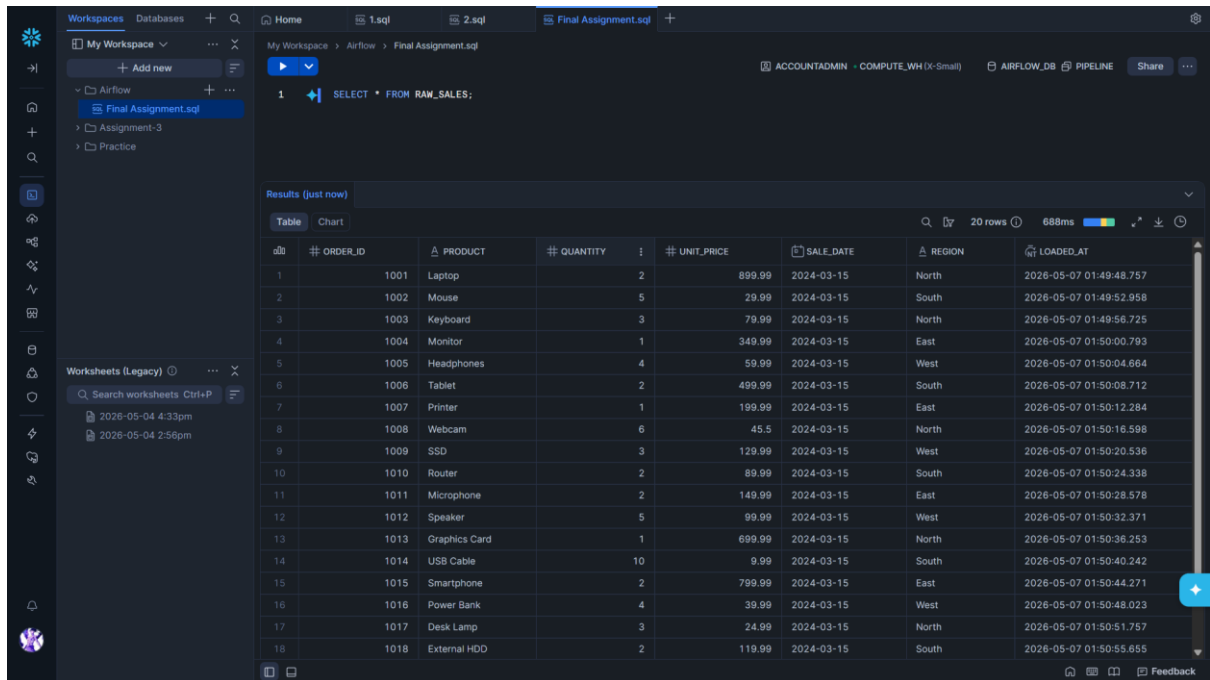
create_table_task >> clear_todays_data_task >> load_csv_task >>
transform_task >> quality_check_task >> print_report_task

```

1. Graph View — all 6 tasks GREEN in correct sequence



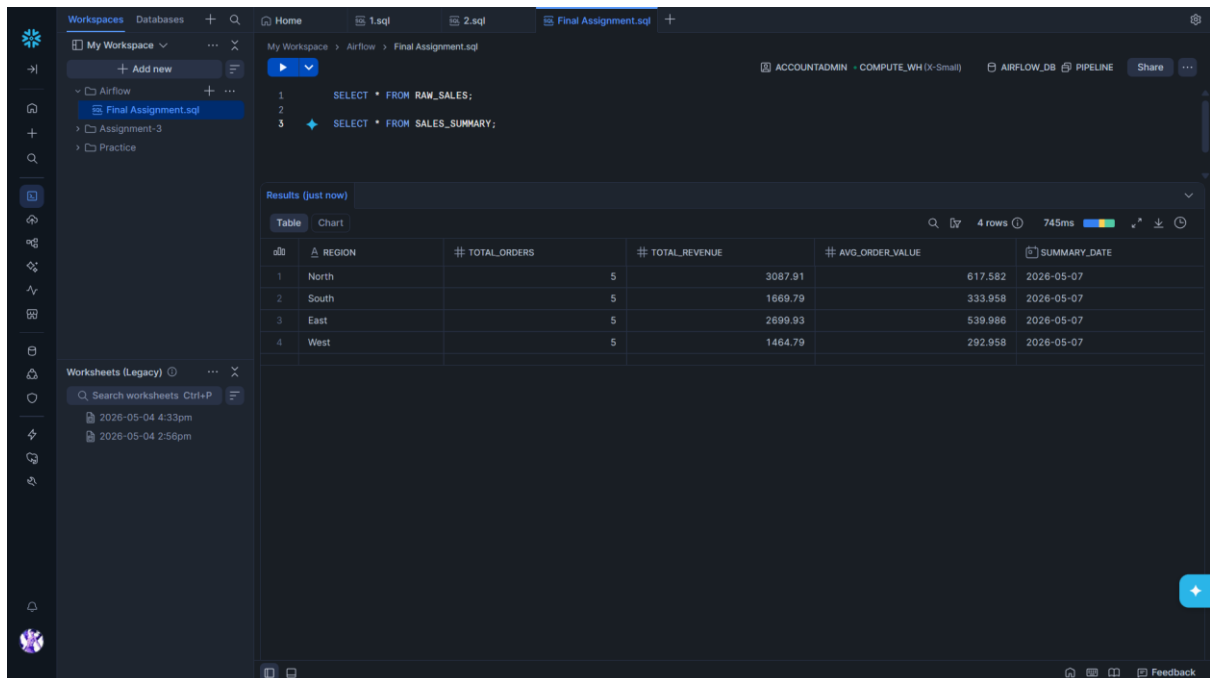
2. Snowflake Worksheet — SELECT * FROM RAW_SALES showing loaded rows with loaded_at column



The screenshot shows a Snowflake worksheet titled "Final Assignment.sql". The SQL query is `SELECT * FROM RAW_SALES;`. The results are displayed in a table with 18 rows and 9 columns. The columns are: ORDER_ID, PRODUCT, QUANTITY, UNIT_PRICE, SALE_DATE, REGION, and LOADED_AT. The data shows various products like Laptop, Mouse, Keyboard, Monitor, Headphones, Tablet, Printer, Webcam, SSD, Router, Microphone, Speaker, Graphics Card, USB Cable, Smartphone, Power Bank, Desk Lamp, and External HDD, all loaded on 2026-05-07.

#	ORDER_ID	PRODUCT	QUANTITY	UNIT_PRICE	SALE_DATE	REGION	LOADED_AT
1	1001	Laptop	2	899.99	2024-03-15	North	2026-05-07 01:49:48.757
2	1002	Mouse	5	29.99	2024-03-15	South	2026-05-07 01:49:52.958
3	1003	Keyboard	3	79.99	2024-03-15	North	2026-05-07 01:49:56.725
4	1004	Monitor	1	349.99	2024-03-15	East	2026-05-07 01:50:00.793
5	1005	Headphones	4	59.99	2024-03-15	West	2026-05-07 01:50:04.664
6	1006	Tablet	2	499.99	2024-03-15	South	2026-05-07 01:50:08.712
7	1007	Printer	1	199.99	2024-03-15	East	2026-05-07 01:50:12.284
8	1008	Webcam	6	45.5	2024-03-15	North	2026-05-07 01:50:16.598
9	1009	SSD	3	129.99	2024-03-15	West	2026-05-07 01:50:20.536
10	1010	Router	2	89.99	2024-03-15	South	2026-05-07 01:50:24.338
11	1011	Microphone	2	149.99	2024-03-15	East	2026-05-07 01:50:28.578
12	1012	Speaker	5	99.99	2024-03-15	West	2026-05-07 01:50:32.371
13	1013	Graphics Card	1	699.99	2024-03-15	North	2026-05-07 01:50:36.253
14	1014	USB Cable	10	9.99	2024-03-15	South	2026-05-07 01:50:40.242
15	1015	Smartphone	2	799.99	2024-03-15	East	2026-05-07 01:50:44.271
16	1016	Power Bank	4	39.99	2024-03-15	West	2026-05-07 01:50:48.023
17	1017	Desk Lamp	3	24.99	2024-03-15	North	2026-05-07 01:50:51.757
18	1018	External HDD	2	119.99	2024-03-15	South	2026-05-07 01:50:55.655

3. Snowflake Worksheet — SELECT * FROM SALES_SUMMARY showing grouped revenue per region



The screenshot shows a Snowflake worksheet titled "Final Assignment.sql". The SQL query is `SELECT * FROM SALES_SUMMARY;`. The results are displayed in a table with 4 rows and 6 columns. The columns are: REGION, TOTAL_ORDERS, TOTAL_REVENUE, AVG_ORDER_VALUE, and SUMMARY_DATE. The data shows grouped revenue for North, South, East, and West regions, all summarized on 2026-05-07.

#	REGION	TOTAL_ORDERS	TOTAL_REVENUE	AVG_ORDER_VALUE	SUMMARY_DATE
1	North	5	3087.91	617.582	2026-05-07
2	South	5	1669.79	333.958	2026-05-07
3	East	5	2699.93	539.986	2026-05-07
4	West	5	1464.79	292.958	2026-05-07

4. Task Log of quality_check — PASS printed for all 3 checks with the actual values shown

The screenshot displays the Airflow web interface. On the left, a sidebar contains navigation links: Home, Dags, Assets, Browse, and Admin. The main canvas shows a DAG named 'daily_sales_loader' with a task 'quality_check' highlighted in a blue box. The task is a 'PythonOperator' and its status is 'success'. To the right, the 'Task Log' for 'quality_check' is shown, indicating 'Success'. The log details include the Operator (PythonOperator), Start Date (2026-05-07 14:21:24), End Date (2026-05-07 14:21:36), Duration (00:00:11.430), and Dag Version (v5). The 'Logs' tab is active, showing a list of log entries. The log content includes a warning about a deprecated attribute, followed by three 'INFO' entries for Snowflake queries. Each query is a 'SELECT COUNT(*)' statement with a different WHERE clause: 'WHERE loaded_at::DATE = CURRENT_DATE()', 'WHERE product IS NULL OR unit_price IS NULL', and 'WHERE unit_price < 0'. Each query is followed by a 'PASS' message indicating the check was successful. The final log entry states 'All quality checks completed successfully' and 'Done. Returned value was: None'.

```
5 [2026-05-07 14:21:25] WARNING - The 'airflow.operators.python.PythonOperator' attribute is deprecated. Please use 'airflow.providers.common.sql.operators.sql.SQLOperator' instead.
6 [2026-05-07 14:21:25] INFO - TaskInstance Details: dag_id=daily_sales_loader task_id=quality_check dagrun_id>manual_2026-05-07
7 [2026-05-07 14:21:26] INFO - Snowflake Connector for Python Version: 4.4.0, Python Version: 3.13.13, Platform: Linux-6.6.87.2-mi
8 [2026-05-07 14:21:26] INFO - Connecting to GLOBAL Snowflake domain
9 [2026-05-07 14:21:28] INFO - Running statement: SELECT COUNT(*)
  FROM RAM_SALES
  WHERE loaded_at::DATE = CURRENT_DATE(); parameters: None
10 [2026-05-07 14:21:28] INFO - Rows affected: 1
11 [2026-05-07 14:21:28] INFO - Snowflake query id: 01c43493-0308-cf06-0027-dcef001f867a
12 [2026-05-07 14:21:28] INFO - PASS: Row count check passed. Rows = 20
13 [2026-05-07 14:21:29] INFO - Snowflake Connector for Python Version: 4.4.0, Python Version: 3.13.13, Platform: Linux-6.6.87.2-mi
14 [2026-05-07 14:21:29] INFO - Connecting to GLOBAL Snowflake domain
15 [2026-05-07 14:21:31] INFO - Running statement: SELECT COUNT(*)
  FROM RAM_SALES
  WHERE product IS NULL
  OR unit_price IS NULL; parameters: None
16 [2026-05-07 14:21:32] INFO - Rows affected: 1
17 [2026-05-07 14:21:32] INFO - Snowflake query id: 01c43493-0308-cf12-0027-dcef0020363e
18 [2026-05-07 14:21:33] INFO - PASS: NULL check passed.
19 [2026-05-07 14:21:33] INFO - Snowflake Connector for Python Version: 4.4.0, Python Version: 3.13.13, Platform: Linux-6.6.87.2-mi
20 [2026-05-07 14:21:33] INFO - Connecting to GLOBAL Snowflake domain
21 [2026-05-07 14:21:34] INFO - Running statement: SELECT COUNT(*)
  FROM RAM_SALES
  WHERE unit_price < 0; parameters: None
22 [2026-05-07 14:21:35] INFO - Rows affected: 1
23 [2026-05-07 14:21:35] INFO - Snowflake query id: 01c43493-0308-cefb-0027-dcef001fb672
24 [2026-05-07 14:21:36] INFO - PASS: Negative price check passed.
25 [2026-05-07 14:21:36] INFO - All quality checks completed successfully.
26 [2026-05-07 14:21:36] INFO - Done. Returned value was: None
```

5. Task Log of print_report — formatted region report with grand total revenue

The screenshot displays the Airflow web interface. On the left, a sidebar contains navigation links: Home, Dags, Assets, Browse, and Admin. The main canvas shows a DAG named 'daily_sales_loader' with a task 'print_report' highlighted in a blue box. The task is a 'PythonOperator' and its status is 'success'. To the right, the 'Task Log' for 'print_report' is shown, indicating 'Success'. The log details include the Operator (PythonOperator), Start Date (2026-05-07 14:21:37), End Date (2026-05-07 14:21:41), Duration (00:00:04.656), and Dag Version (v5). The 'Logs' tab is active, showing a list of log entries. The log content includes a 'Snowflake query id' entry, followed by a 'SALES REPORT' header. The report is divided into four sections, one for each region: North, South, East, and West. Each section lists 'Total Orders', 'Total Revenue', and 'Average Order Value'. The final log entry shows the 'GRAND TOTAL REVENUE' as 8922.42 and 'END REPORT'. The final log entry states 'Done. Returned value was: None'.

```
11 [2026-05-07 14:21:40] INFO - Snowflake query id: 01c43493-0308-c941-0027-dcef0020846e
12 [2026-05-07 14:21:41] INFO - ===== SALES REPORT =====
13 [2026-05-07 14:21:41] INFO - Region: North
  Total Orders: 5
  Total Revenue: 3087.91
  Average Order Value: 617.582
14 [2026-05-07 14:21:41] INFO - Region: South
  Total Orders: 5
  Total Revenue: 1669.79
  Average Order Value: 333.95799999999997
15 [2026-05-07 14:21:41] INFO - Region: East
  Total Orders: 5
  Total Revenue: 2699.9300000000001
  Average Order Value: 539.9800000000001
16 [2026-05-07 14:21:41] INFO - Region: West
  Total Orders: 5
  Total Revenue: 1464.79
  Average Order Value: 292.95799999999997
17 [2026-05-07 14:21:41] INFO - GRAND TOTAL REVENUE: 8922.42
18 [2026-05-07 14:21:41] INFO - ===== END REPORT =====
19 [2026-05-07 14:21:41] INFO - Done. Returned value was: None
```